OBJECT™ TECHNOLOGIES

Exception Handling

www.object.co.in

- What is exception
- Nature of exceptions
- What is exception handling
- Exception class hiearchy
- Exception types
- Common scenarios
- Using try-catch
- Use of finally

What is Exception

OBJECT

- exception is shorthand for the phrase "exceptional event."
- An exception is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions and the program/Application terminates abnormally
- An exception can occur for many different reasons. Following are some scenarios where an exception occurs.
  - A user has entered an invalid data.
  - A file that needs to be opened cannot be found.
  - A network connection has been lost in the middle of communications or the JVM has run out of memory.

The Nature of Exceptions

OBJECT

Broadly speaking, there are three different situations that cause exceptions to be thrown:

- **Exceptions due to programming errors:** In this category, exceptions are generated due to programming errors (e.g., NullPointerException and IllegalArgumentException).
- **Exceptions due to client code errors:** Client code attempts something not allowed by the API, and thereby violates its contract. The client can take some alternative course of action, if there is useful information provided in the exception. For example: an exception is thrown while parsing an XML document that is not well-formed. The exception contains useful information about the location in the XML document that causes the problem. The client can use this information to take recovery steps.
- **Exceptions due to resource failures:** Exceptions that get generated when resources fail. For example: the system runs out of memory or a network connection fails. The client's response to resource failures is context-driven. The client can retry the operation after some time or just log the resource failure and bring the application to a halt.

Exception Handling

OBJECT

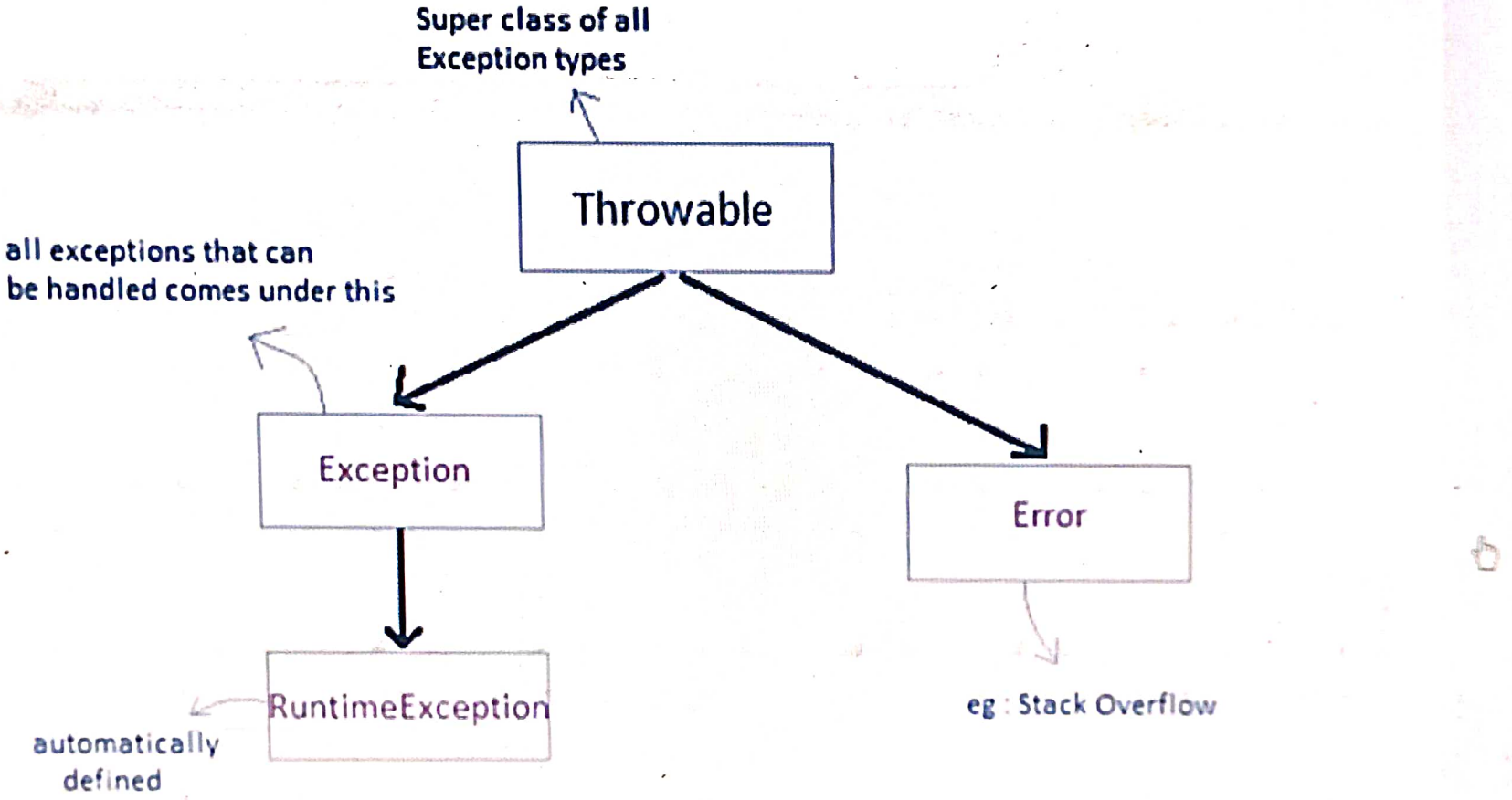
- Exception Handling is the mechanism to handle runtime malfunctions. We need to handle such exceptions to prevent abrupt termination of program
- When an exceptional events occurs in java, an exception is said to be thrown. The code that's responsible for doing something about the exception is called an **exception handler**.
- Exception handling is not responsible for preventing exceptions but it is a defensive programming where in programmer tries to handle the situation so that execution is not halted

Advantages of Exception Handling

OBJECT

- Exception handling allows us to control the normal flow of the program by using exception handling in program.
- It throws an exception whenever a calling method encounters an error providing that the calling method takes care of that error.
- It also gives us the scope of organizing and differentiating between different error types using a separate block of codes. This is done with the help of try-catch blocks.
- If an exception is raised, which has not been handled by programmer then program execution can get terminated and system prints a non user friendly error message.

Exception class Hierarchy



Exception class Hierarchy

OBJECT

- Throwable : This class is at the top of the hierarchy. All exceptions are sublcass of Throwable. This class adds the characteristic of 'can be thrown' for any instance of Throwable .
- Exception : It is a subclass of Throwable. Exceptions are expected to be handled by the programmers.
- Error : It is a subclass of Throwable. Errors are not expected to be handled by the programmers. Their occurrence is rare and they are more fatal in nature
- RuntimeException : It is a subclass of Exception. Exceptions inherited from RuntimeException may or may not be handled by the programmers

Exception Types

OBJECT

- **Checked Exception**

  The exception that can be predicted by the programmer. *Example* : File that need to be opened is not found. These type of exceptions must be checked at compile time. All classes that directly inherit from Exception are checked exceptions.

- **Unchecked Exception**

  Unchecked exceptions are the class that extends RuntimeException. Unchecked exception are ignored at compile time. *Example :* ArithmeticException, NullPointerException, Array Index out of Bound exception. Unchecked exceptions are checked at runtime.

Common scenarios where exceptions may occur

1) Scenario where ArithmeticException occurs

If we divide any number by zero, there occurs an ArithmeticException.

```
int a=50/0;//ArithmeticException
```

2) Scenario where NullPointerException occurs

If we have null value in any variable, performing any operation by the variable occurs an NullPointerException.

```
String s=null;
System.out.println(s.length());//NullPointerException
```

3) Scenario where NumberFormatException occurs

The wrong formatting of any value, may occur NumberFormatException. Suppose I have a string variable that have characters, converting this variable into digit will occur NumberFormatException.

```
String s="abc";
int i=Integer.parseInt(s);//NumberFormatException
```

4) Scenario where ArrayIndexOutOfBoundsException occurs

If you are inserting any value in the wrong index, it would result ArrayIndexOutOfBoundsException as shown below:

```
int a[]=new int[5];
a[10]=50; //ArrayIndexOutOfBoundsException
```

Keywords

OBJECT

There are 5 keywords used in java exception handling.

- try
- catch
- finally
- throw
- throws

try-catch

OBJECT

- Java try block is used to enclose the code that might throw an exception.
- It must be used within the method.
- Java try block must be followed by either catch or finally block or by both.
- Catch is at the end of try block to handle the exceptions.
- We can have multiple catch blocks with a try and try-catch block can be nested also.
- Catch block requires a parameter that should be of type Exception.

```
try{
//code that may throw exception
}catch(Exception_class_Name ref){}
```

```
try{
//code that may throw exception
}finally{}
```

Flow of try catch block

OBJECT

- If an exception occurs in try block then the control of execution is passed to the catch block from try block. The exception is caught up by the corresponding catch block. A single try block can have multiple catch statements associated with it, but each catch block can be defined for only one exception class.
- If the try block is not **throwing any exception**, the catch block will be completely ignored and the program continues.
  If the try block **throws an exception**, the appropriate catch block (if one exists) will catch it
- All the statements in the catch block will be executed and then the program continues.

Uncaught Exceptions

OBJECT

```
class UncaughtException
{
 public static void main(String args[])
 {
  int a = 0;
  int b = 7/a;      // Divide by zero, will lead to exception
 }
}
```

- When exception is not handled using try and catch block , it will lead to an exception at runtime, hence the Java run-time system will construct an exception and then throw it.
- As we don't have any mechanism for handling exception in the above program, hence the default handler will handle the exception and will print the details of the exception on the terminal

Finally block

OBJECT

- A **finally block** may be associated with a **try block**. It identifies a block of statements that needs to be executed regardless of whether or not an **exception occurs** within the try block.
- After all other try-catch processing is complete, the **code inside the finally block executes**. It is not mandatory to include a finally block at all, but if you do, it will run regardless of whether an exception was thrown and handled by the try and catch parts of the block.
- In normal execution the finally block is executed after try block. When any exception occurs first the catch block is executed and then finally block is executed.
- An exception in the finally block, exactly **behaves like any other exception**.
- The code present in the **finally block** executes even if the try or catch block contains control transfer statements like **return**, break or continue.

Cases when the finally block doesn't execute

The circumstances that prevent execution of the code in a finally block are:

- The death of a Thread
- Using of the System. exit() method.
- Due to an exception arising in the finally block.

Example : try-catch-finally

OBJECT

```
class Example2{
   public static void main(String args[]){
      try{
         System.out.println("First statement of try block");
         int num=45/0;
         System.out.println(num);
      }
      catch(ArrayIndexOutOfBoundsException e){
         System.out.println("ArrayIndexOutOfBoundsException");
      }
      finally{
         System.out.println("finally block");
      }
      System.out.println("Out of try-catch-finally block");
   }
}
```

- Output :

```
First statement of try block
finally block
Exception in thread "main" java.lang.ArithmeticException: / by ze
        at beginnersbook.com.Example2.main(Details.java:6)
```